

LOGIC CIRCUITS





LOGIC CIRCUITS

- Binary Boolean algebra provides a concise way to express the operation of a logic circuit formed by a combination of logic gates so that the output can be determined for various combinations of input values.
- The basic elements used to build electronic circuits are called basic gates and they implement one of the Boolean operations: $\wedge$, $\vee$, $\neg$.
- A gate takes the values from its input wires and combines them with the appropriate Boolean operation to produce the label on its output wire.
- A Boolean function models the functionality of a logic circuit.
The variables used in the function's expression are represented by the inputs of the circuit. Each wire in the circuit represents some part of the expression. The final output of the whole circuit represents the function's expression.

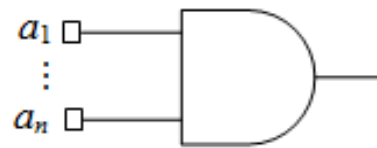


BASIC GATES – IEEE STANDARDS

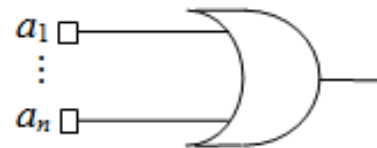
gate	symbol for the gate	Boolean operation, symbols used
AND		conjunction $a \wedge b$, $a \cap b$, $a * b$, ab
OR		disjunction $a \vee b$ $a \cup b$, $a + b$
NOT		negation $\neg a$, $\bar{a}$



The AND and OR gates can be generalized to have more input variables:

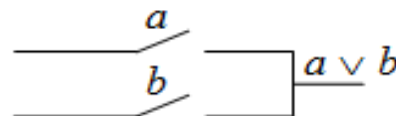


$$a_1 \wedge a_2 \wedge \dots \wedge a_n \text{ OR } a_1 a_2 \dots a_n$$

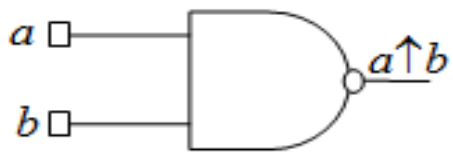
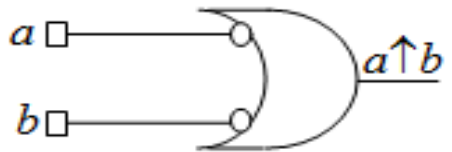
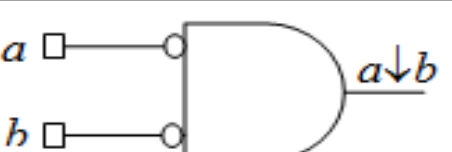


$$a_1 \vee a_2 \vee \dots \vee a_n$$

- At the physical level electronic circuits are composed of a power supply connection and contacts.
- A contact is described by a Boolean variable having one of the values: 0 for an open contact or 1 for a closed contact.
- The AND, OR gates correspond to a series connection and to a parallel connection respectively.



DERIVED GATES: XOR, NAND, NOR, NXOR

gate	symbol	Boolean function	alternative symbol
XOR		$a \oplus b = \bar{a}b \vee a\bar{b}$	
NAND		$a \uparrow b = \overline{ab}$ $= \bar{a} \vee \bar{b}$	
NOR		$a \downarrow b = \overline{a \vee b}$ $= \bar{a} \bar{b}$	
NXOR		$a \otimes b = \bar{\bar{a}}\bar{b} \vee ab$ $= (\bar{a} \vee b)(\bar{b} \vee a)$	

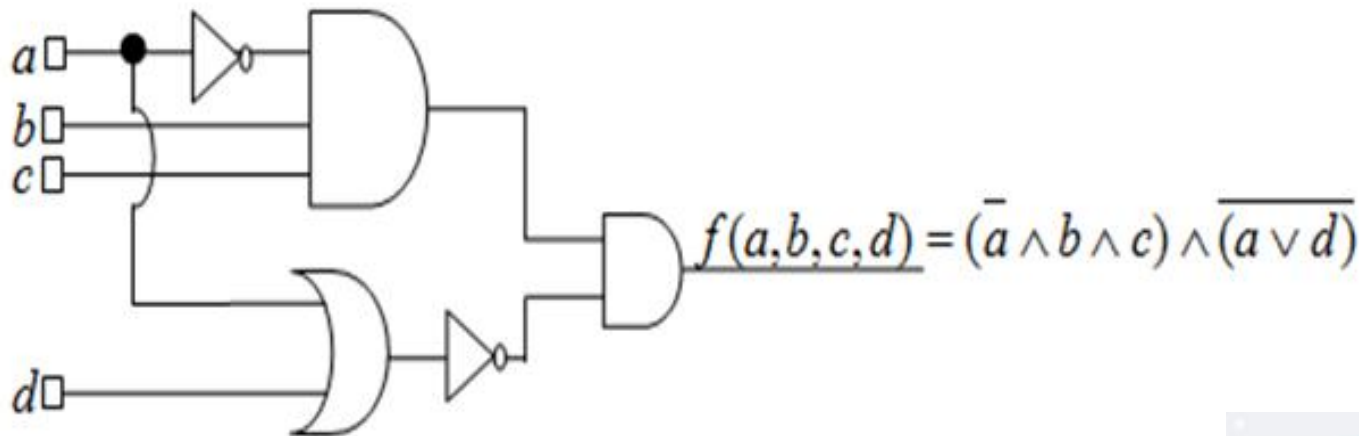
Note: The **AND**, **OR**, **NAND**, **NOR**, **NXOR** gates may have any number of inputs, but practical commercial gates are mostly limited to 2, 3 and 4 inputs to fit the standard IC (integrated circuits) packages.



EXAMPLE 1

A logic circuit can be implemented directly from the function's expression.

Given the Boolean function $f(a,b,c,d) = (\bar{a} \wedge b \wedge c) \wedge \overline{(a \vee d)}$ of four variables, we draw the logic circuit having f as an output.



EXAMPLE 2

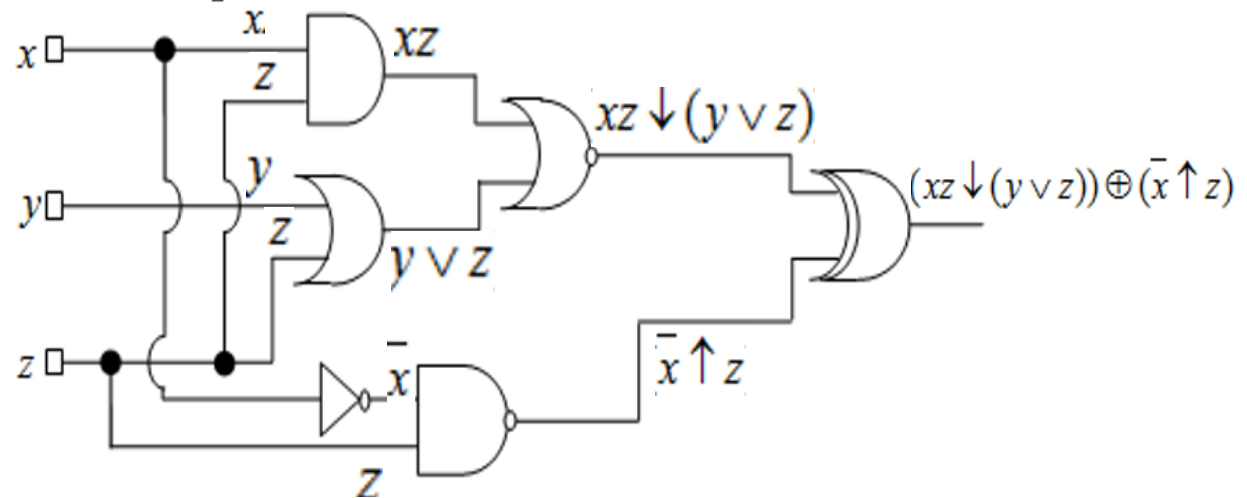


Logic circuit analysis

- the task of determining the functionality of a given logic circuit from its diagram.

To derive the Boolean expression for a given logic circuit, begin in the diagram at the left-most inputs and work toward the final output, writing the expression for each gate.

Let us consider the logic circuit depicted below:

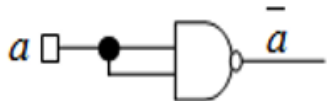
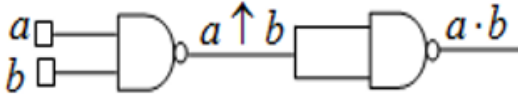
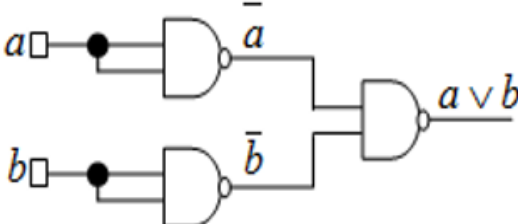
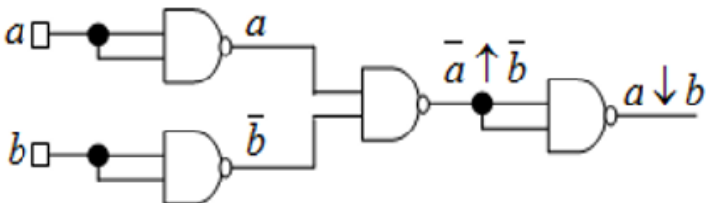


The Boolean expression which models its functionality is $f(x,y,z) = (xz \downarrow (y \vee z)) \oplus (\bar{x} \uparrow z)$



EXAMPLE 3. NAND IS A UNIVERSAL GATE

- IT CAN BE USED TO PRODUCE ALL THE OTHER GATES

$\bar{a} = \overline{a \cdot a} = a \uparrow a$	
$a \cdot b = \overline{\overline{a \cdot b}} = \overline{a \uparrow b} = (a \uparrow b) \uparrow (a \uparrow b)$	
$a \vee b = \overline{\overline{a \vee b}} = \overline{a \uparrow b} = (a \uparrow a) \uparrow (b \uparrow b)$	
$a \downarrow b = \overline{a \vee b} = \overline{(a \uparrow b)} = ((a \uparrow a) \uparrow (b \uparrow b)) \uparrow ((a \uparrow a) \uparrow (b \uparrow b))$	

Note: NOR gate is a universal gate.

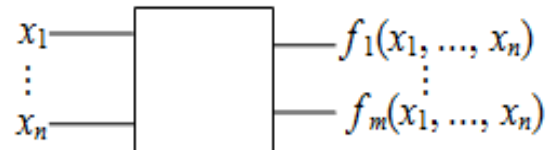




COMBINATIONAL CIRCUITS

Combinational circuit

- a logic circuit which does not use any memory and whose outputs respond immediately to the inputs
- it can have m number of outputs.
- each output corresponds to a Boolean function.



An elementary logic circuit is a particular case of a combinational circuit ($m = 1$).

Combinational logic circuit synthesis task:

- *input*: the problem specification describing the functionality of the desired circuit
- *output*: the implementation of the logic circuit.
- *to solve the task*: first we build the truth tables defining the functions, then we simplify them and finally we build the circuit using the logic gates.

EXAMPLES OF USEFUL COMBINATIONAL CIRCUITS

Some of the combinational circuits used in the hardware of computers:

- comparator of two binary digits
- adder (half and full) of two binary digits
- subtractor (half and full) of two binary digits
- encoder used for the binary codification of the decimal digits
- decoder converts a group of 4 binary digits into a decimal digit



EXAMPLE 4

The comparator circuit checks the relation between two binary digits.

The truth tables of the output functions:

x_1	x_2	<i>equal</i> ($x_1 = x_2$)	<i>smaller</i> ($x_1 < x_2$)	<i>bigger</i> ($x_1 > x_2$)
0	0	1	0	0
0	1	0	1	0
1	0	0	0	1
1	1	1	0	0

- smaller or equal: $x_1 \leq x_2$,
 $seq(x_1, x_2) = \bar{x}_1 \vee x_2$
- bigger or equal: $x_1 \geq x_2$,
 $beq(x_1, x_2) = x_1 \vee \bar{x}_2$

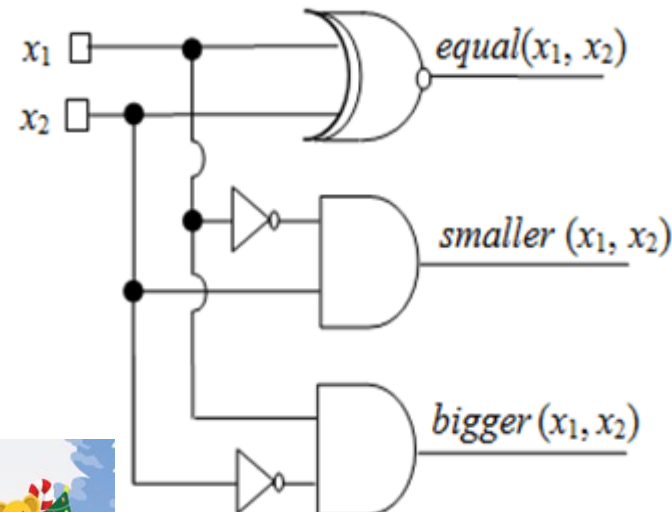
Boolean expressions for the output functions:

$$equal(x_1, x_2) = \bar{x}_1 \bar{x}_2 \vee x_1 x_2 = x_1 \otimes x_2$$

$$smaller(x_1, x_2) = \bar{x}_1 x_2$$

$$bigger(x_1, x_2) = x_1 \bar{x}_2$$

Combinational circuit:

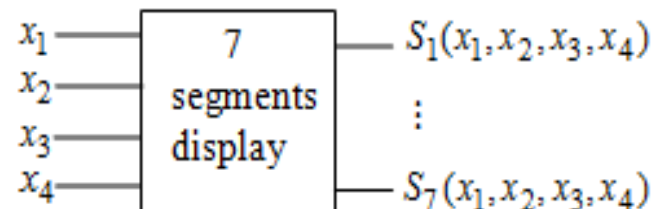
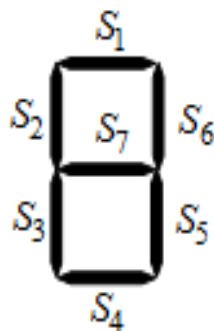


EXAMPLE 5. 7 SEGMENTS CIRCUIT

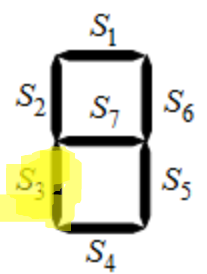
Electronic display of the decimal digits using 7 segments (LED):



- inputs: 4 variables
- outputs: 7 functions corresponding to the segments.
- the combinations of 4 binary digits which do not correspond to a decimal digit will generate an emission of light only for the segment S_7 .



EXAMPLE 5. 7 SEGMENTS CIRCUIT



<i>Decimal digit</i>	<i>Inputs</i>				<i>Outputs</i>						
	x_1	x_2	x_3	x_4	S_1	S_2	S_3	S_4	S_5	S_6	S_7
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	0	0	0	1	1	0
2	0	0	1	0	1	0	1	1	0	1	1
3	0	0	1	1	1	0	0	1	1	1	1
4	0	1	0	0	0	1	0	0	1	1	1
5	0	1	0	1	1	1	0	1	1	0	1
6	0	1	1	0	1	1	1	1	1	0	1
7	0	1	1	1	1	0	0	0	1	1	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	0	1	1	1	1
A	1	0	1	0	0	0	0	0	0	0	1
B	1	0	1	1	0	0	0	0	0	0	1
C	1	1	0	0	0	0	0	0	0	0	1
D	1	1	0	1	0	0	0	0	0	0	1
E	1	1	1	0	0	0	0	0	0	0	1
F	1	1	1	1	0	0	0	0	0	0	1



EXAMPLE 5. IMPLEMENT THE SIMPLIFIED LOGIC CIRCUIT FOR S_3 USING ONLY NAND GATES

$$DCF(S_3) = m_0 \vee m_2 \vee m_6 \vee m_8$$

Karnaugh diagram:

$x_1x_2 \backslash x_3x_4$	00	01	11	10
00	m_0			m_2
01				m_6
11				
10	m_8			

Simple factorizations $\Rightarrow$ maximal monoms

$$\max_1 = \overline{x_1}\overline{x_2}\overline{x_4} = m_0 \vee m_2$$

$$\max_2 = \overline{x_1}\overline{x_3}\overline{x_4} = m_2 \vee m_6$$

$$\max_3 = \overline{x_2}\overline{x_3}\overline{x_4} = m_0 \vee m_8$$

$$M(S_3) = \{ \max_1, \max_2, \max_3 \}.$$

$$C(S_3) = \{ \max_2, \max_3 \} \neq M(S_3).$$

This is the second case of the simplification algorithm, but the central monoms cover all the minterms.

A unique disjunctive simplified form of S_3 :

$$S_3^{DS} = \max_2 \vee \max_3 = \overline{x_1}\overline{x_3}\overline{x_4} \vee \overline{x_2}\overline{x_3}\overline{x_4}$$



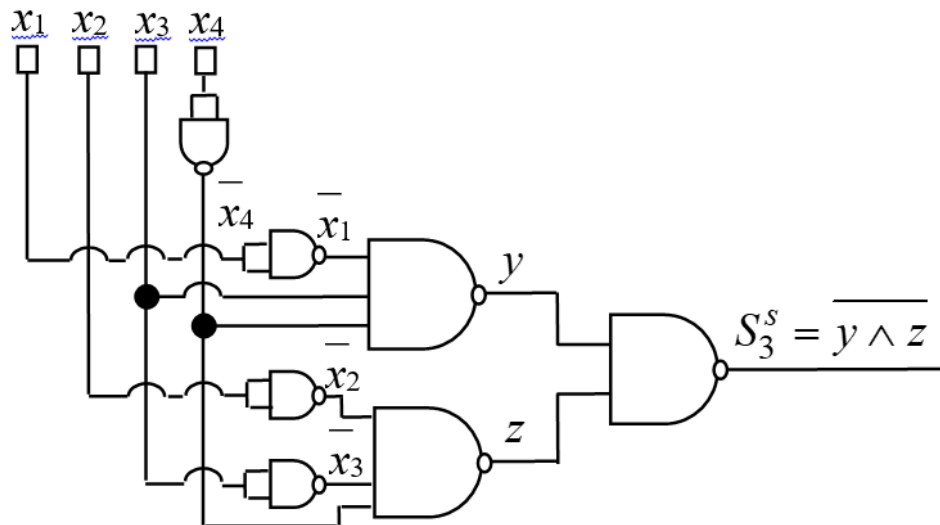
EXAMPLE 5(CONTD.)

We double negate the expression of S_3^{DS} and we transform it.

$$\begin{aligned} S_3^{DS} &= (\bar{x}_1 \wedge x_3 \wedge \bar{x}_4) \vee (\bar{x}_2 \wedge \bar{x}_3 \wedge \bar{x}_4) = \\ &= \overline{\overline{(\bar{x}_1 \wedge x_3 \wedge \bar{x}_4) \vee (\bar{x}_2 \wedge \bar{x}_3 \wedge \bar{x}_4)}}} = \overline{\overline{(\bar{x}_1 \wedge x_3 \wedge \bar{x}_4)} \wedge \overline{\overline{(\bar{x}_2 \wedge \bar{x}_3 \wedge \bar{x}_4)}}} = \\ &= \overline{\overline{(x_1 \wedge x_1)} \wedge x_3 \wedge \overline{(x_4 \wedge x_4)}} \wedge \overline{\overline{(x_2 \wedge x_2)} \wedge \overline{(x_3 \wedge x_3)} \wedge \overline{(x_4 \wedge x_4)}} \end{aligned}$$

The logic circuit has:

- four NAND gates which implement the negations of the variables ($\bar{x}_1 = \overline{x_1 \wedge x_1}$)
- two NAND gates, with 3 inputs each, for the expressions:
 $y = \overline{(x_1 \wedge x_1) \wedge x_3 \wedge (x_4 \wedge x_4)}$ and $z = \overline{(x_2 \wedge x_2) \wedge (x_3 \wedge x_3) \wedge (x_4 \wedge x_4)}$
- a two-input NAND gate corresponding to $S_3^{DS} = \overline{y \wedge z}$



$$a \uparrow b = \overline{a \wedge b}$$

$$\bar{a} = \overline{a \wedge a} = a \uparrow a$$



EXAMPLE 5. IMPLEMENT THE LOGIC CIRCUIT FOR S_3 USING ONLY NOR GATES

Dual simplification algorithm:

$$CCF(S_3) = M_1 \wedge M_3 \wedge M_4 \wedge M_5 \wedge M_7 \wedge M_9 \wedge \\ \wedge M_{10} \wedge M_{11} \wedge M_{12} \wedge M_{13} \wedge M_{14} \wedge M_{15}$$

Karnaugh diagram:

$x_1x_2 \backslash x_3x_4$	00	01	11	10
00		M_1	M_3	
01	M_4	M_5	M_7	
11	M_{12}	M_{13}	M_{15}	M_{14}
10		M_9	M_{11}	M_{10}

One triple dual factorization:

$$\text{maxd}_1 = M_1 \wedge M_3 \wedge M_5 \wedge M_7 \wedge M_{13} \wedge M_{15} \wedge M_9 \wedge M_{11} = \bar{x}_4$$

Three double dual factorizations:

$$\text{maxd}_2 = M_4 \wedge M_5 \wedge M_{12} \wedge M_{13} = \bar{x}_2 \vee x_3$$

$$\text{maxd}_3 = M_{15} \wedge M_{14} \wedge M_{11} \wedge M_{10} = \bar{x}_1 \vee \bar{x}_3$$

$$\text{maxd}_4 = M_{12} \wedge M_{13} \wedge M_{15} \wedge M_{14} = \bar{x}_1 \vee \bar{x}_2$$

$Md(S_3) = \{ \text{maxd}_1, \text{maxd}_2, \text{maxd}_3, \text{maxd}_4 \}$ - the set of maximal disjunctions

$Cd(S_3) = \{ \text{maxd}_1, \text{maxd}_2, \text{maxd}_3 \}$ - the set of central disjunctions

The central disjunctions cover all the maxterms, so there is a unique conjunctive simplified form:

$$S_3^{CS} = \text{maxd}_1 \wedge \text{maxd}_2 \wedge \text{maxd}_3 = \bar{x}_4 \wedge (\bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_3)$$

EXAMPLE 5(CONTD.)

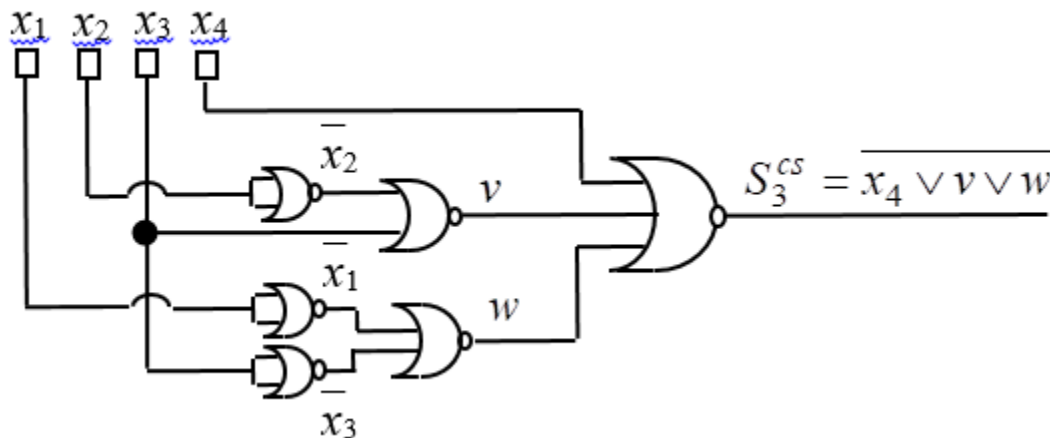


We double negate the expression of S_3^{cs} and we transform it.

$$\begin{aligned} S_3^{cs} &= \bar{x}_4 \wedge (\bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_3) = \overline{\overline{\bar{x}_4 \wedge (\bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_3)}} \\ &= \overline{\overline{\bar{x}_4} \vee \overline{(\bar{x}_2 \vee x_3)} \vee \overline{(\bar{x}_1 \vee \bar{x}_3)}} = \\ &= \overline{x_4 \vee \overline{(x_2 \vee x_2)} \vee x_3 \vee \overline{(x_1 \vee x_1)} \vee \overline{(x_3 \vee x_3)}} \end{aligned}$$

In the implementation of the logic circuit, six NOR gates are used as follows:

- three gates for $\bar{x}_1, \bar{x}_2, \bar{x}_3$ (ex: $\bar{x}_1 = \overline{x_1 \vee x_1}$);
- two gates, with two inputs each, for the expressions:
 $v = \overline{(x_2 \vee x_2) \vee x_3}$ and $w = \overline{(x_1 \vee x_1) \vee (x_3 \vee x_3)}$;
- a final gate with three inputs corresponding to $S_3^{cs} = \overline{x_4 \vee v \vee w}$



$$\begin{aligned} a \downarrow b &= \overline{a \vee b} \\ \bar{a} &= \overline{a \vee a} = a \downarrow a \end{aligned}$$



Merry Christmas

Let the beauty of
the season fill you
with warmth and
happiness!





I wish you Health...
So you may enjoy each day in comfort.

I wish you the Love of friends and family...
And Peace within your heart.

I wish you the Beauty of nature...
That you may enjoy the work of God.

I wish you Wisdom to choose priorities...
For those things that really matter in life.

I wish you Generosity so you may share...
All good things that come to you.

I wish you Happiness and Joy...
And Blessings for the New Year.

I wish you the best of everything...
That you so well deserve.



EXAMPLE 6. FULL ADDER

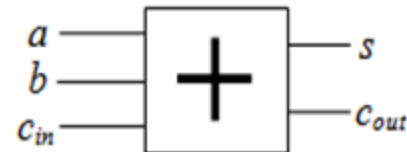
Input variables: a, b - *two digits* (of the same rank in a binary number)

c_{in} - the *input carry* digit from the previous rank (less significant)

Output functions: s - the *one-digit sum* of a and b and c_{in} ,

c_{out} - the generated carry digit, called *output carry*

Inputs			Outputs	
a	b	c_{in}	$s =$ $a + b +$ c_{in}	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



The output functions have the following disjunctive canonical forms:

$$s(a, b, c_{in}) = \bar{a}\bar{b}c_{in} \vee \bar{a}b\bar{c}_{in} \vee a\bar{b}\bar{c}_{in} \vee abc_{in}$$

$$c_{out}(a, b, c_{in}) = \bar{a}bc_{in} \vee a\bar{b}c_{in} \vee ab\bar{c}_{in} \vee abc_{in}$$



EXAMPLE 6. FULL ADDER (CONTD.)

Veitch diagrams are used in the simplification of the Boolean functions S and c_{out} .

$$s(a, b, c_{in})$$

	c_{in}	$\bar{c}_{in}$	
a	1	1	
$\bar{a}$		1	1
	b	$\bar{b}$	b

$$c_{out}(a, b, c_{in})$$

	c_{in}	$\bar{c}_{in}$	
a	1	1	1
$\bar{a}$	1		
	b	$\bar{b}$	b

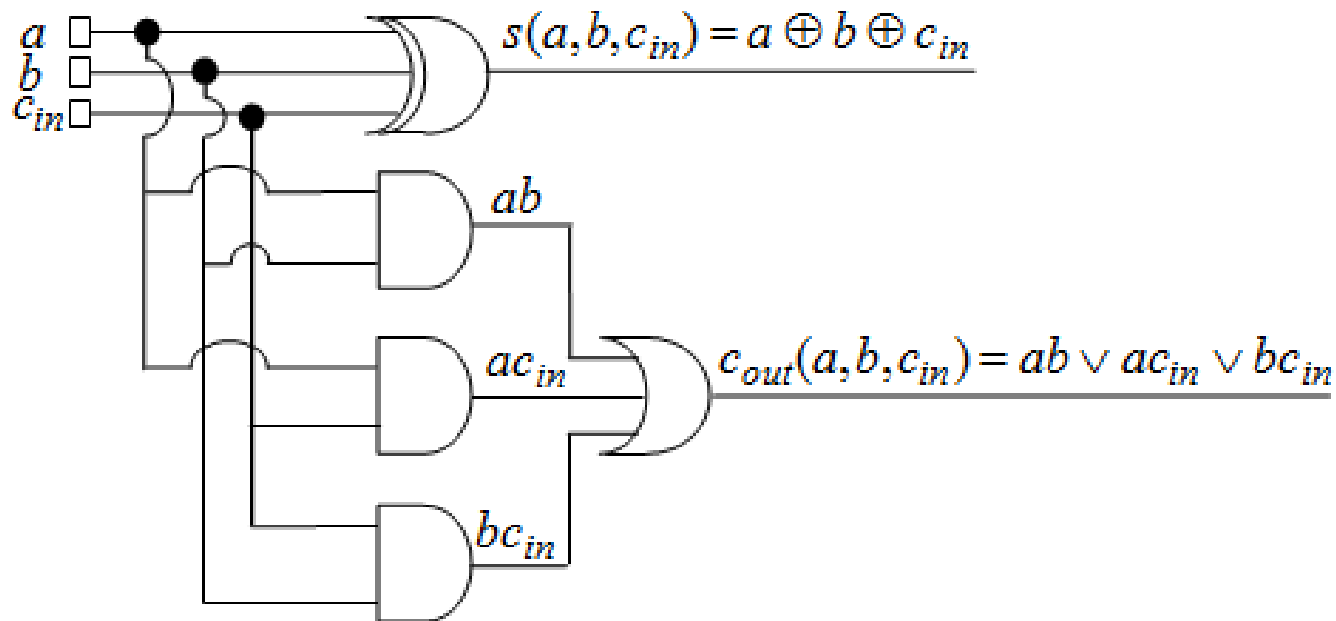
- the function $s(a, b, c_{in}) = \bar{a}\bar{b}c_{in} \vee \bar{a}b\bar{c}_{in} \vee a\bar{b}\bar{c}_{in} \vee abc_{in}$ is in a disjunctive simplified form
- for the function S there is a simplified form containing derived gates
with fewest gates: $s(a, b, c_{in}) = a \oplus b \oplus c_{in}$

(S is 1 when there is an odd number of values 1 for the input variables).

- the simplified form of c_{out} using only basic gates is $c_{out}(a, b, c_{in}) = ab \vee ac_{in} \vee bc_{in}$.

EXAMPLE 6. FULL ADDER (CONTD.)

The circuit diagram is depicted below:

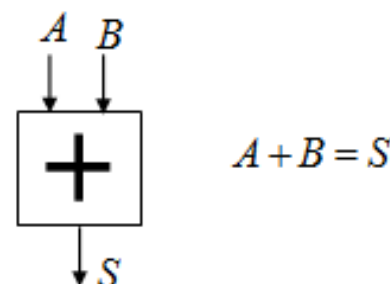


Example 7. n - bits adder

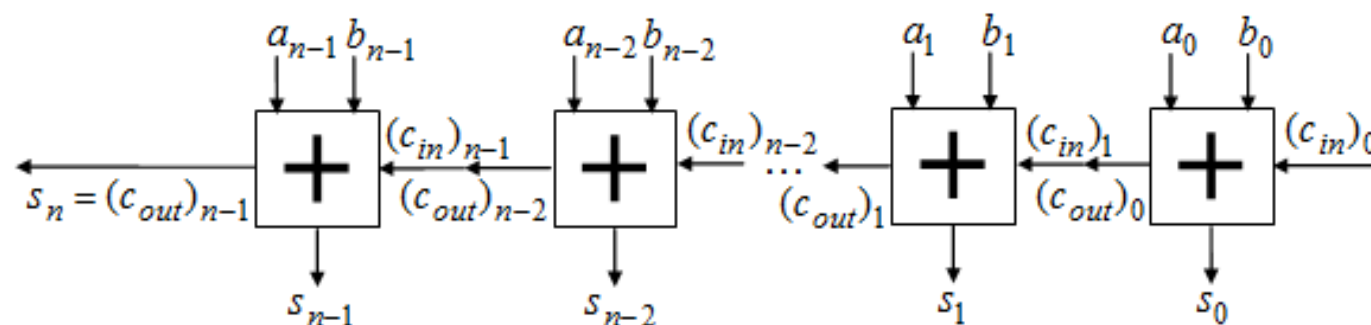
- used to add two n -bit binary numbers
- it is composed of n full adders in cascade,

$$A = (a_{n-1}a_{n-2}\dots a_0)_{(2)} , B = (b_{n-1}b_{n-2}\dots b_0)_{(2)} \text{ and } S = A + B , S = (s_n s_{n-1}\dots s_0)_{(2)} .$$

The logic circuit is symbolized as:



The combinational circuit is composed of n full adders:



Example 7. n – bits adder (contd.)

For $A = 10011011_{(2)}$ and $B = 11001111_{(2)}$, $S = A + B = 101101010_{(2)}$.

Eight full adders in cascade are used to perform this addition as follows:

carries	$(c_{out})_7$ $= s_8$ $= 1$	$(c_{out})_6$ $= (c_{in})_7$ $= 0$	$(c_{out})_5$ $= (c_{in})_6$ $= 0$	$(c_{out})_4$ $= (c_{in})_5$ $= 1$	$(c_{out})_3$ $= (c_{in})_4$ $= 1$	$(c_{out})_2$ $= (c_{in})_3$ $= 1$	$(c_{out})_1$ $= (c_{in})_2$ $= 1$	$(c_{out})_0$ $= (c_{in})_1$ $= 1$	$(c_{in})_0$ $= 0$
A		$a_7 = 1$	$a_6 = 0$	$a_5 = 0$	$a_4 = 1$	$a_3 = 1$	$a_2 = 0$	$a_1 = 1$	$a_0 = 1$
B		$b_7 = 1$	$b_6 = 1$	$b_5 = 0$	$b_4 = 0$	$b_3 = 1$	$b_2 = 1$	$b_1 = 1$	$b_0 = 1$
$S =$ $A + B$	$s_8 = 1$	$s_7 = 0$	$s_6 = 1$	$s_5 = 1$	$s_4 = 0$	$s_3 = 1$	$s_2 = 0$	$s_1 = 1$	$s_0 = 0$

□



FUN WITH LOGIC CIRCUITS

Domino computer:

<https://www.youtube.com/watch?v=w6E7aQnA4Ws>

Marble adding machine

<https://www.youtube.com/watch?v=GcDshWmhF4A>

Water computer:

<https://www.youtube.com/watch?v=lxXaizglscw>

Logic gates using Gears, Marbles & Dominoes:

<https://www.youtube.com/watch?v=kQeDnFUNW-Q>

A Computer That Runs on Marbles:

<https://www.youtube.com/watch?v=8BOvLL8ok8I>



EXAMPLE 8. FULL SUBTRACTOR

The **full subtractor** is a combinational circuit with three inputs and two outputs.

Input variables:

a - the *minuend bit*, b - the *subtrahend bit*, of the same rank in a binary number

c_{in} - the *input borrow* digit from the previous rank (less significant)

Output functions: d - the one-digit *difference* of a and b ,

c_{out} - the *output borrow*

The table of truth values is presented below.

Inputs			Outputs	
a	b	c_{in}	d	c_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



If $a - b - c_{in} \geq 0$

then

$$d = a - b - c_{in} \text{ and } c_{out} = 0$$

else

$$d = 2 + a - b - c_{in} \text{ and } c_{out} = 1$$



EXAMPLE 8. FULL SUBTRACTOR (CONTD.)

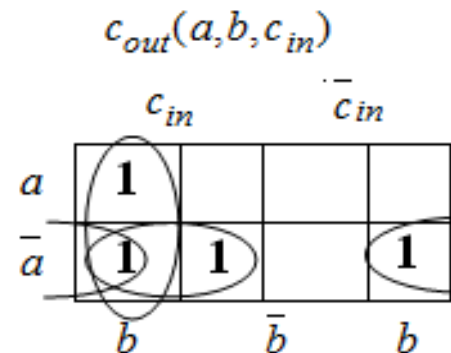
The output functions have the following disjunctive canonical forms:

$$d(a, b, c_{in}) = \bar{a}\bar{b}c_{in} \vee \bar{a}b\bar{c}_{in} \vee a\bar{b}\bar{c}_{in} \vee abc_{in}$$

$$c_{out}(a, b, c_{in}) = \bar{a}\bar{b}c_{in} \vee \bar{a}b\bar{c}_{in} \vee a\bar{b}c_{in} \vee abc_{in}$$

To be noted that the function d is the same as the output function s from the full adder circuit.

The function c_{out} is simplified using Veitch diagram:



The simplified forms are:

$$d(a, b, c_{in}) = a \oplus b \oplus c_{in}$$

$$c_{out}(a, b, c_{in}) = \bar{a}b \vee \bar{a}c_{in} \vee bc_{in}$$



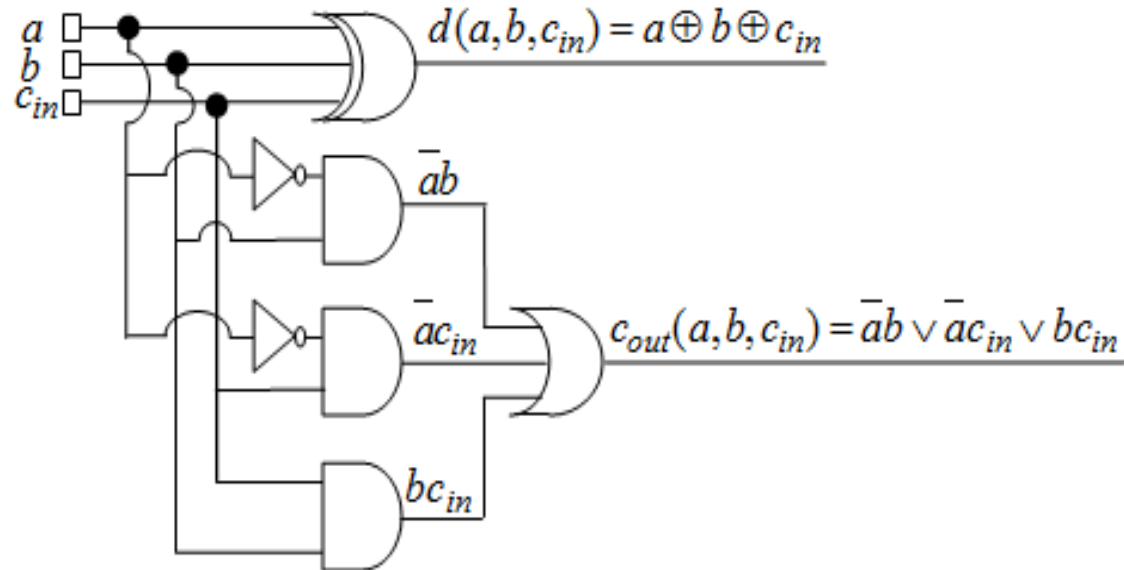
EXAMPLE 8. FULL SUBTRACTOR (CONTD.)

The simplified forms are:

$$d(a, b, c_{in}) = a \oplus b \oplus c_{in}$$

$$c_{out}(a, b, c_{in}) = \bar{a}b \vee \bar{a}c_{in} \vee bc_{in}$$

The circuit diagram is depicted below:



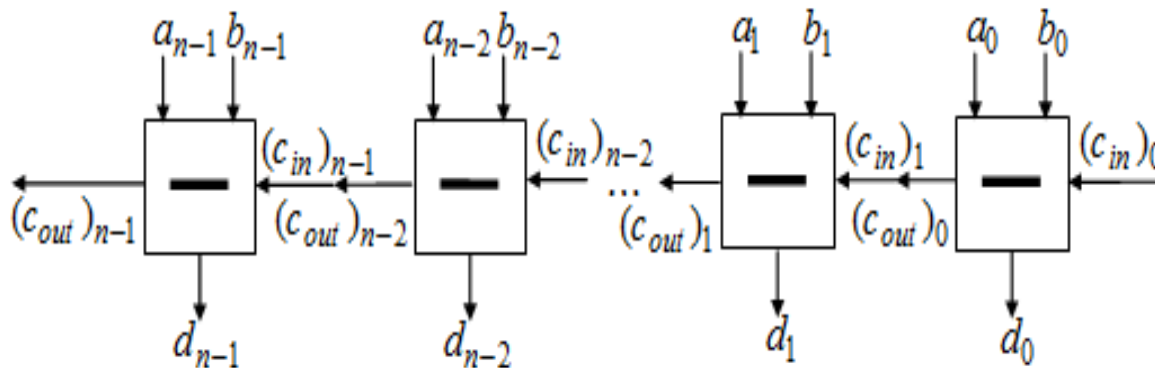
Example 9. n - bits subtractor

- used to subtract two n -bit binary numbers
- composed of n full subtractors in cascade,

Let $A = (a_{n-1}a_{n-2}\dots a_0)_{(2)}$ be the minuend and $B = (b_{n-1}b_{n-2}\dots b_0)_{(2)}$ be the subtrahend.

$D = A - B$, $D = (d_{n-1}d_{n-2}\dots d_0)_{(2)}$ is calculated using n full subtractors in cascade.

The circuit diagram of n - bit subtractor:



The output borrow of the previous full subtractor is connected to the input borrow of the next full subtractor.



Example 9. n – bits subtractor (contd.)

For $A = 11101010_{(2)}$ and $B = 10011111_{(2)}$, $D = A - B = 1001011_{(2)}$

borrows $(c_{out})_7$ $= 0$	$(c_{out})_6$ $= (c_{in})_7$ $= 0$	$(c_{out})_5$ $= (c_{in})_6$ $= 0$	$(c_{out})_4$ $= (c_{in})_5$ $= 1$	$(c_{out})_3$ $= (c_{in})_4$ $= 1$	$(c_{out})_2$ $= (c_{in})_3$ $= 1$	$(c_{out})_1$ $= (c_{in})_2$ $= 1$	$(c_{out})_0$ $= (c_{in})_1$ $= 1$	$(c_{in})_0$ $= 0$
A	$a_7 = 1$	$a_6 = 1$	$a_5 = 1$	$a_4 = 0$	$a_3 = 1$	$a_2 = 0$	$a_1 = 1$	$a_0 = 0$
B	$b_7 = 1$	$b_6 = 0$	$b_5 = 0$	$b_4 = 1$	$b_3 = 1$	$b_2 = 1$	$b_1 = 1$	$b_0 = 1$
$D =$ $A - B$	$d_7 = 0$	$d_6 = 1$	$d_5 = 0$	$d_4 = 0$	$d_3 = 1$	$d_2 = 0$	$d_1 = 1$	$d_0 = 1$



EXAMPLE 10. ENCODER

- the combinational circuit used for the binary codification of the decimal digits
- 10 input variables: $x_0, \dots, x_9$, corresponding to the decimal digits
- 4 output functions: f_1, f_2, f_3, f_4 - representing the corresponding four binary digits
- the variable $x_i = 1$ only when we want to convert the decimal digit i into binary.

	Inputs										Outputs			
decimal digit	x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	f_1	f_2	f_3	f_4
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0	1
2	0	0	1	0	0	0	0	0	0	0	0	0	1	0
3	0	0	0	1	0	0	0	0	0	0	0	0	1	1
4	0	0	0	0	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	1	0	0	0	0	0	1	0	1
6	0	0	0	0	0	0	1	0	0	0	0	1	1	0
7	0	0	0	0	0	0	0	1	0	0	0	1	1	1
8	0	0	0	0	0	0	0	0	1	0	1	0	0	0
9	0	0	0	0	0	0	0	0	0	1	1	0	0	1

EXAMPLE 10. ENCODER (CONTD.)

The disjunctive simplified forms of the Boolean functions are:

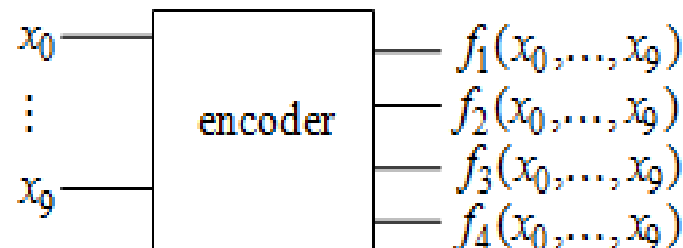
$$f_1(x_0, \dots, x_9) = x_8 \vee x_9$$

$$f_2(x_0, \dots, x_9) = x_4 \vee x_5 \vee x_6 \vee x_7$$

$$f_3(x_0, \dots, x_9) = x_2 \vee x_3 \vee x_6 \vee x_7$$

$$f_4(x_0, \dots, x_9) = x_1 \vee x_3 \vee x_5 \vee x_7 \vee x_9$$

The corresponding combinational circuit is:

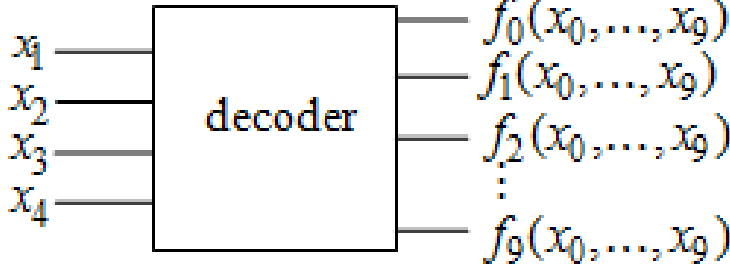


EXAMPLE 11. DECODER

- converts a group of 4 binary digits into a decimal digit
- $f_i(x_1, x_2, x_3, x_4) = 1$ only for $x_1x_2x_3x_4_{(2)} = i_{(10)}, i = \overline{0, 9}$.
- $f_7(0,1,1,1) = 1, 0111_{(2)} = 7_{(10)}$, for all the other arguments the function is 0.

Inputs				Outputs										disjunctive canonical form
x_1	x_2	x_3	x_4	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	
0	0	0	0	1	0	0	0	0	0	0	0	0	0	$f_0(x_1, x_2, x_3, x_4) = \overline{x_1} \overline{x_2} \overline{x_3} \overline{x_4}$
0	0	0	1	0	1	0	0	0	0	0	0	0	0	$f_1(x_1, x_2, x_3, x_4) = \overline{x_1} \overline{x_2} \overline{x_3} x_4$
0	0	1	0	0	0	1	0	0	0	0	0	0	0	$f_2(x_1, x_2, x_3, x_4) = \overline{x_1} \overline{x_2} x_3 \overline{x_4}$
0	0	1	1	0	0	0	1	0	0	0	0	0	0	$f_3(x_1, x_2, x_3, x_4) = \overline{x_1} \overline{x_2} x_3 x_4$
0	1	0	0	0	0	0	0	1	0	0	0	0	0	$f_4(x_1, x_2, x_3, x_4) = \overline{x_1} x_2 \overline{x_3} \overline{x_4}$
0	1	0	1	0	0	0	0	0	1	0	0	0	0	$f_5(x_1, x_2, x_3, x_4) = \overline{x_1} x_2 \overline{x_3} x_4$
0	1	1	0	0	0	0	0	0	0	1	0	0	0	$f_6(x_1, x_2, x_3, x_4) = \overline{x_1} x_2 x_3 \overline{x_4}$
0	1	1	1	0	0	0	0	0	0	0	1	0	0	$f_7(x_1, x_2, x_3, x_4) = \overline{x_1} x_2 x_3 x_4$
1	0	0	0	0	0	0	0	0	0	0	0	1	0	$f_8(x_1, x_2, x_3, x_4) = x_1 \overline{x_2} \overline{x_3} \overline{x_4}$
1	0	0	1	0	0	0	0	0	0	0	0	0	1	$f_9(x_1, x_2, x_3, x_4) = x_1 \overline{x_2} \overline{x_3} x_4$

EXAMPLE 11. DECODER (CONTD.)

Output Boolean functions	Combinational circuit
$f_0(x_1, x_2, x_3, x_4) = \overline{x_1} \overline{x_2} \overline{x_3} \overline{x_4}$ $f_1(x_1, x_2, x_3, x_4) = \overline{x_1} \overline{x_2} \overline{x_3} x_4$ $f_2(x_1, x_2, x_3, x_4) = \overline{x_1} \overline{x_2} x_3 \overline{x_4}$ $f_3(x_1, x_2, x_3, x_4) = \overline{x_1} \overline{x_2} x_3 x_4$ $f_4(x_1, x_2, x_3, x_4) = \overline{x_1} x_2 \overline{x_3} \overline{x_4}$ $f_5(x_1, x_2, x_3, x_4) = \overline{x_1} x_2 \overline{x_3} x_4$ $f_6(x_1, x_2, x_3, x_4) = \overline{x_1} x_2 x_3 \overline{x_4}$ $f_7(x_1, x_2, x_3, x_4) = \overline{x_1} x_2 x_3 x_4$ $f_8(x_1, x_2, x_3, x_4) = x_1 \overline{x_2} \overline{x_3} \overline{x_4}$ $f_9(x_1, x_2, x_3, x_4) = x_1 \overline{x_2} \overline{x_3} x_4$	



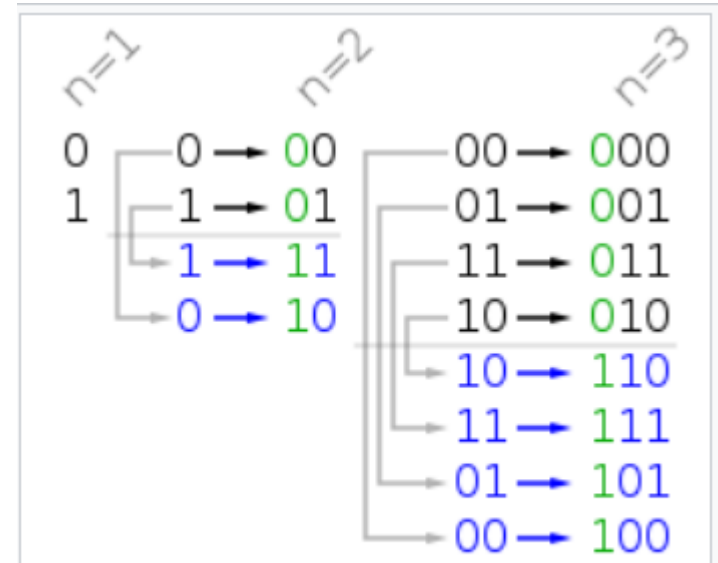
BINARY CODES

- they are used to represent the ten decimal digits on 4 bits.
- there exist *weighted codes* and *unweighted codes*.
- in a *weighted code* each binary digit has a 'weight' and the decimal value is obtained as the weighted sum of all 4 bits.
- *BCD - Binary Coded Decimal* is an example of a weighted code, with the weights for the digits: 8,4,2,1 (from the most significant digit to the least significant one).
- the most used *unweighted codes* are
 - *Excess 3* (obtained by adding '0011' to each *BCD* word)
 - *Gray code* (a cyclic code with the property that all the successive words differ by a single digit).



GRAY CODES ON N BITS

The binary-reflected Gray code list for n bits can be generated recursively from the list for $n - 1$ bits by reflecting the list (i.e. listing the entries in reverse order), prefixing the entries in the original list with a binary 0, prefixing the entries in the reflected list with a binary 1.



The first few steps of the reflect-and-prefix method.

Gray Codes are now widely used in digital communications, such as digital terrestrial television and some cable television systems, to aid in error correction and improve the signal's overall quality.



BINARY CODES (CONTD.)



M.Lupea

Decimal value	<i>BCD</i>				<i>Gray code</i>			
	b_3 8	b_2 4	b_1 2	b_0 1	g_3	g_2	g_1	g_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0



EXAMPLE 12.

Implement the combinational circuits for converting:

from *BCD* to *Gray* code;



from *Gray* code to *BCD*;

